



Documentação de projeto: Micro-ondas
Cliente: INE5411

Florianópolis, 30 de Junho de 2026



Sumário

1. Informações Gerais.....	3
Objetivo.....	3
Escopo do Trabalho realizado.....	3
2. Detalhamento do projeto.....	4
Planejamento.....	4
Manual de Simulação do Firmware: Importância e Diretrizes.....	9
3. Considerações finais.....	12

1. INFORMAÇÕES GERAIS

Objetivo

Este documento tem como objetivo descrever o processo de desenvolvimento e o modo de operação do projeto de firmware de um micro-ondas, conforme solicitado pela equipe técnica do cliente INE5411. Este projeto tem como funcionalidades a definição manual de um timer que será, ao ativar o micro-ondas, decrementado a cada segundo, além da funcionalidade de pausar o timer e, se desejado, zerar o contador, além dessas funcionalidades o usuário tem a opção de abrir ou fechar a porta.

Escopo do Trabalho realizado

O serviço prestado compreende o desenvolvimento e implementação de um firmware, escrito em linguagem *Assembly* para o processador MIPS, destinado a realizar as funcionalidades de possibilitar a definição por parte do usuário de um tempo, de 00 até 99, o qual iniciará um processo de decremento no momento em que o usuário ativar o devido sistema, enquanto o timer estiver sendo decrementado, o usuário tem a possibilidade de pausar seu decremento utilizando a tecla de *pause/reset*, pressioná-la novamente limpa o timer e define-o novamente para “00”. Outra funcionalidade possível é a de abrir/fechar a porta, se a porta for aberta o contador, se estava funcionando, é pausado, o display exibe a mensagem “OP” e é impossível reiniciar o timer até que a porta seja fechada, se a porta estiver aberta e o usuário pressionar o botão de *pause/reset* o tempo será redefinido para o valor “00” novamente. O documento de especificação foi fornecido pela equipe técnica do cliente, orientando as implementações necessárias. Assim, o escopo do trabalho contratado inclui:

- Elaboração do projeto do firmware para ser embarcado em um micro-ondas, atendendo aos requisitos do documento de especificação fornecido pela equipe técnica do cliente;
- Implementação do firmware em linguagem *Assembly* para o processador MIPS, com o *part number* especificado pela equipe técnica do cliente;
- Simulação das funcionalidades implementadas para validação do funcionamento, utilizando o simulador MARS como ferramenta;
- Elaboração de documentação técnica que descreve o processo de planejamento e implementação do firmware; e
- Entrega dos arquivos-fonte do projeto.

2. DETALHAMENTO DO PROJETO

Planejamento

O firmware desenvolvido visa controlar as funcionalidades principais de um micro-ondas, as quais foram listadas nas seções anteriores, vale destacar a adoção de uma abordagem de projeto estruturada do tipo *top-down* e totalmente orientada a componentes. A concepção do sistema partiu da abstração das regras de negócio de alto nível do eletrodoméstico para, em seguida, segmentar as responsabilidades em sub módulos isolados. Essa separação em múltiplos arquivos (.asm) assegura a boa manutenibilidade e separação clara de responsabilidades do sistema.

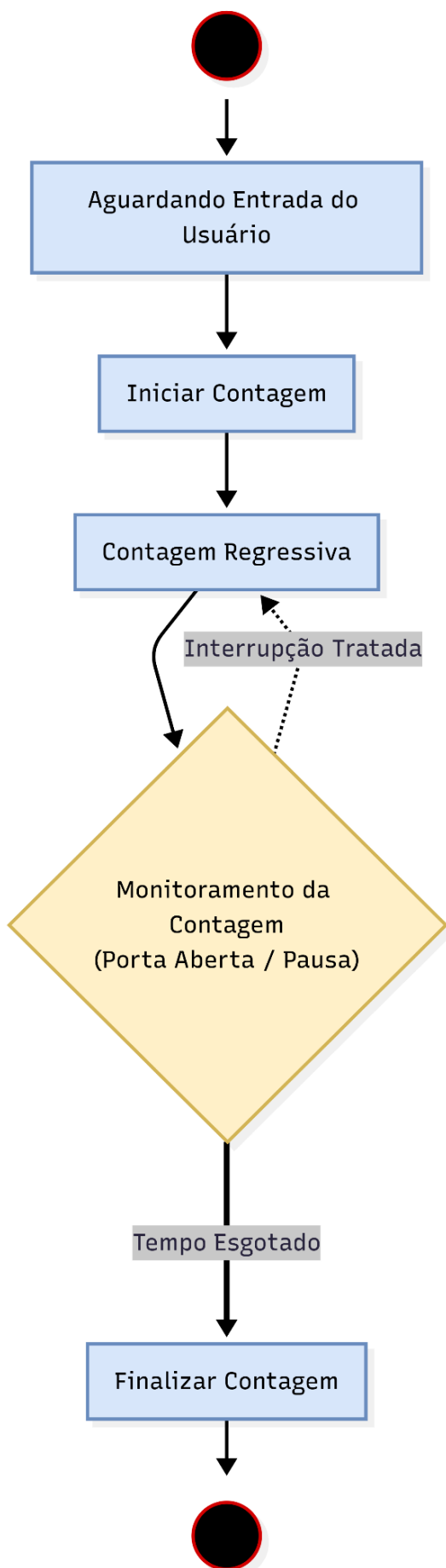


Figura 1: Diagrama de atividades

A Figura 1 ilustra o fluxo de funcionamento do projeto. Os **retângulos azuis** representam as ações diretas e sequenciais executadas pelo sistema ou pelo usuário, enquanto o **losango amarelo** indica um ponto condicional de monitoramento contínuo. O avanço padrão do processo é guiado pelas **setas contínuas**, e a **seta pontilhada** demonstra o fluxo de retorno automático após o sistema tratar uma interrupção temporária (como a pausa ou a abertura da porta), retomando a contagem de onde parou.

Para garantir a organização, escalabilidade e a manutenibilidade do código Assembly, a arquitetura do projeto foi dividida estruturalmente em exatos cinco módulos. Essa decisão de design foi estritamente guiada pelo Princípio de Responsabilidade Única (*Single Responsibility Principle*), assegurando que cada arquivo gerencie apenas um aspecto específico do sistema, seja ele controle de hardware, contagem de tempo ou regra de negócio.

A arquitetura do sistema foi mapeada e implementada através dos seguintes componentes fundamentais:

- **main.asm**: Módulo de inicialização e loop principal, atua como o ponto de entrada do programa. Sua responsabilidade restringe-se a invocar as rotinas de inicialização das tabelas de decodificação e das variáveis de estado, entrando imediatamente em um ciclo infinito de execução que coordena a atualização contínua do sistema mediante as interações do usuário e as rotinas necessárias.
- **logica.asm**: Centraliza a lógica fundamental do sistema por meio de uma Máquina de Estados Finitos (FSM). Ele gerencia as variáveis globais de tempo e estado atual, avalia as entradas geradas pelo teclado, valida as condições de segurança, como o status da porta, e coordena o fluxo de transição entre os diferentes modos de operação.
- **teclado.asm**: Abstrai a interface com o teclado matricial do simulador *Digital Lab Sim*. Realiza a leitura física dos registradores de linha e coluna mapeados em memória, unifica a leitura de coordenadas e faz o tratamento de *debounce* lógico (armazenando a tecla anterior) para evitar capturas duplicadas e um mesmo clique.
- **display.asm**: Responsável por encapsular o controle dos displays de 7 segmentos. Ele contém a tabela com os padrões de bits necessários para renderizar caracteres numéricos e alfanuméricos. Realiza também a decomposição matemática do tempo, a divisão por 10, para isolar dezenas e unidades, enviando os dados aos endereços de memória corretos de cada display de forma transparente para os demais módulos.
- **timer.asm**: Prevê o sincronismo de um segundo exigido pelo cronômetro sem causar o travamento do fluxo principal. Utiliza a chamada de sistema *syscall 30* para ler o relógio interno do sistema operacional e calcular a diferença de tempo de maneira a não bloquear outras ações.

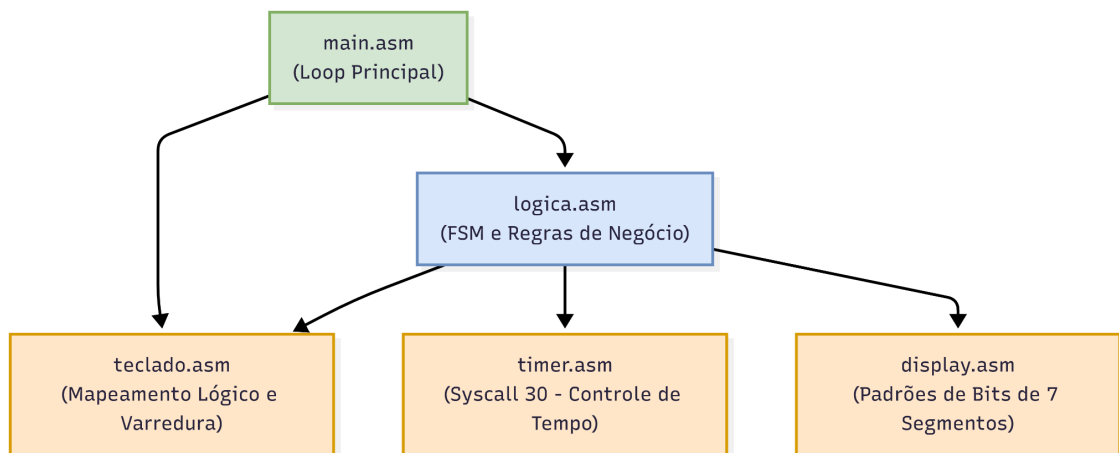


Figura 2: Diagrama de comunicação entre módulos (grafo de dependências dos .asm)

A Figura 2 ilustra a arquitetura de software e a dependência hierárquica do projeto. O bloco **verde** representa o orquestrador e ponto de entrada (*main.asm*), o bloco **azul** identifica o núcleo de processamento e controle de estados (*logica.asm*), e os blocos **amarelos** destacam os módulos periféricos de suporte e drivers de hardware (*teclado.asm*, *timer.asm* e *display.asm*), demonstrando visualmente a aplicação do princípio de responsabilidade única.

Modelo Comportamental da Máquina de Estados

Para garantir o estrito cumprimento dos requisitos de controle e segurança do micro-ondas, o comportamento lógico foi modelado como uma máquina de estados finitos contida no arquivo *logica.asm*. O sistema alterna de maneira previsível e por meio de uma variável de controle entre cinco estados:

- **IDLE:** É o estado inicial. O sistema permanece aguardando que o usuário digite o tempo desejado através das teclas numéricas. Conforme os dígitos entram, o valor do temporizador é atualizado e exibido. O acionamento da tecla A (ligar) só é aceito se o tempo configurado for estritamente maior que zero e a porta estiver fechada (gerenciada por outro estado).
- **RUNNING:** Desencadeado a partir do estado IDLE após a validação do tempo e da porta. Neste estado, o temporizador decrementa uma unidade a cada intervalo de 1000 milissegundos validado pelo componente de tempo. O display é atualizado dinamicamente. Se a tecla B for pressionada, o sistema transita para o modo pausado, já se a tecla C for pressionada, o processo é interrompido e vai para o modo de porta aberta.
- **PAUSED:** Interrompe temporariamente a contagem regressiva, congelando o tempo atual no display. Se o usuário pressionar a tecla A novamente (e a porta estiver fechada), o sistema retoma o estado RUNNING de onde parou. Se a tecla B (parar/cancelar) for pressionada pela segunda vez consecutiva, o tempo é zerado e o sistema retorna ao estado IDLE.

- **DOOR_OPEN:** Pode ser acessado a partir de qualquer estado sempre que a tecla C (sensor de porta) for acionada para abrir a porta. Caso ocorra durante o estado RUNNING, a contagem é suspensa imediatamente. O display passa a exibir a mensagem fixa "OP". Enquanto a porta estiver aberta, qualquer comando para ligar é ignorado. Se o usuário pressionar a tecla B neste modo, o contador é limpo para "00", mas a mensagem "OP" segue sendo emitida no display até que a porta seja fechada.
- **DONE:** Ativado assim que o contador regressivo atinge o valor zero no modo RUNNING. Este estado executa uma rotina de sinalização visual que faz o display piscar o valor "00" por três vezes consecutivas em intervalos regulares de tempo. Concluído o ciclo de alertas, as variáveis são limpas e o controle retorna automaticamente para o modo IDLE.

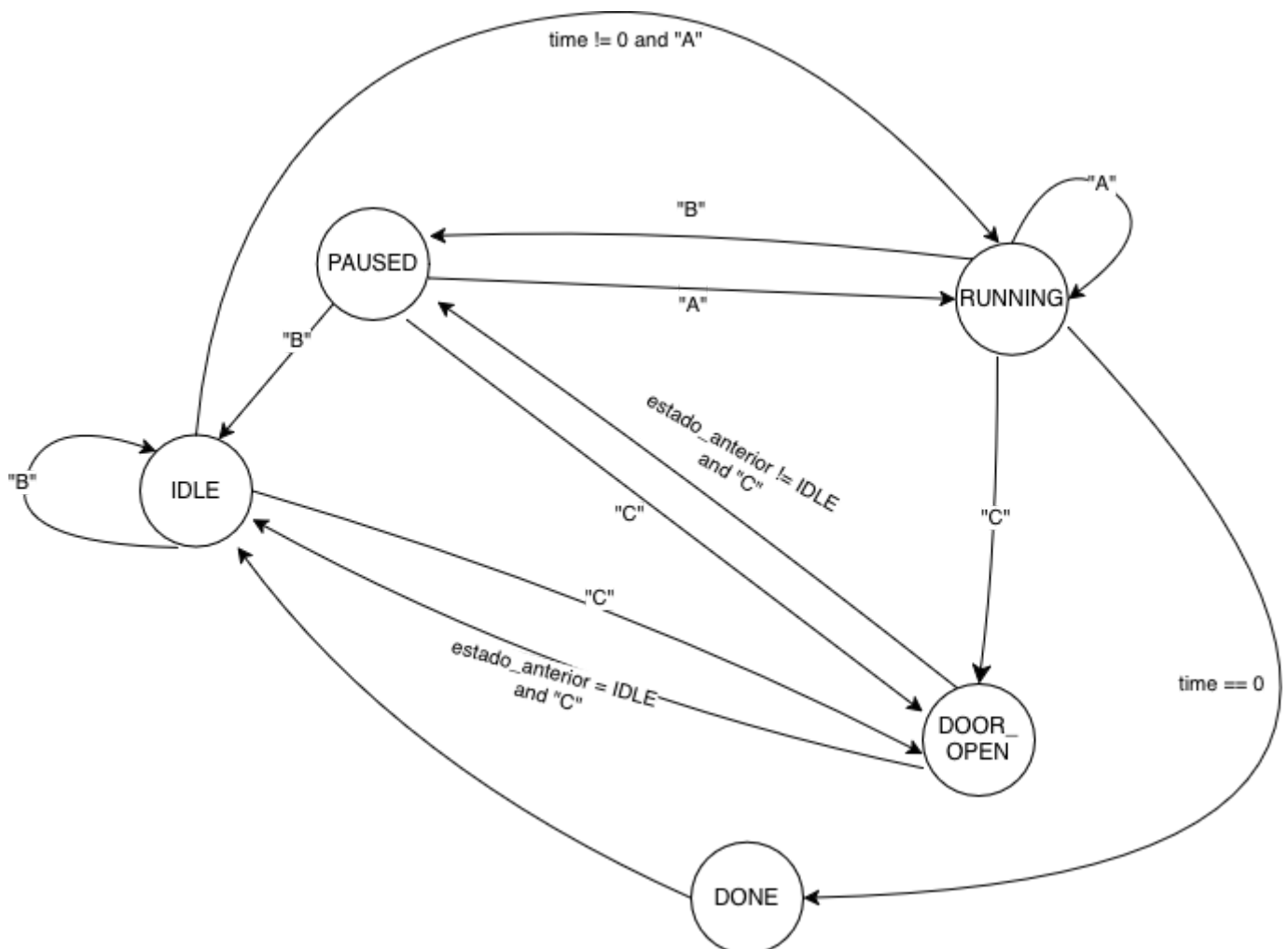


Figura 3: Diagrama da Máquina de Estados Finitos do Firmware desenvolvido, apresenta os 5 estados e suas possíveis transições



A Figura 3 mapeia o comportamento dinâmico do sistema, onde cada nodo representa um estado lógico funcional (IDLE, RUNNING, PAUSED, DOOR_OPEN e DONE) e as linhas direcionais especificam as transições exatas acionadas pelos inputs do usuário (teclas A, B e C) ou por eventos de temporização (*timer*), garantindo o controle e a segurança do simulador.

Sincronismo e Fluxo de Execução Não-Bloqueante

O sincronismo do sistema foi resolvido por meio de um fluxo de *polling* em *main.asm*. A cada iteração do loop principal, o estado do teclado é lido e processado imediatamente. Em paralelo, o módulo de temporização verifica de forma assíncrona se a diferença entre o tempo atual do sistema operacional e o registro do último segundo atingiu a marca de 1000ms. Se a condição for falsa, o programa ignora o decremento e prossegue na varredura; se for verdadeira, o sinalizador é ativado, o tempo é decrementado dentro de *logica.asm* e o display é atualizado. Essa estratégia assegura uma taxa de amostragem alta para as entradas do usuário, garantindo precisão temporal e resposta quase imediata dos mecanismos.

Manual de Simulação do Firmware: Importância e Diretrizes

Este manual fornece as instruções detalhadas para a correta inicialização, operação e validação do firmware do Micro-ondas. O objetivo é garantir que a equipe técnica possa replicar os testes de maneira consistente, cobrindo possíveis cenários de uso e mecanismos de segurança especificados.

1. Preparação do Ambiente de Teste

Para que o sistema funcione corretamente, a arquitetura modular do projeto exige que todos os arquivos estejam no mesmo diretório.

- **Passo 1:** Certifique-se de que todos os arquivos fonte (*main.asm*, *logica.asm*, *timer.asm*, *teclado.asm*, *display.asm*) estão salvos na mesma pasta.
- **Passo 2:** Abra o simulador e carregue exclusivamente o arquivo ***main.asm***.
- **Passo 3:** Abra a ferramenta **Digital Lab Sim**.
- **Passo 4:** Conecte o Digital Lab Sim ao MIPS (“Conect to MIPS”) e, em seguida, compile e execute o código no simulador principal.

2. Interface e Comandos

O sistema utiliza o teclado hexadecimal e os displays de 7 segmentos do Digital Lab Sim para interação. Os comandos estão mapeados da seguinte forma:



- **Teclas de 0 a 9:** Entrada de dados numéricos para definir o tempo em segundos (de 00 a 99).
- **Tecla A (Ligar/Start):** Inicia a contagem regressiva e simula o aquecimento. Só é aceito se o tempo for maior que zero e a porta estiver fechada.
- **Tecla B (Parar/Cancelar):** Durante o funcionamento, um clique pausa a contagem. Se o sistema já estiver pausado ou no modo de espera (IDLE), um clique zera o tempo no painel.
- **Tecla C (Sensor da Porta):** Funciona como um interruptor (toggle). Um clique simula a abertura da porta e o clique seguinte simula o fechamento.

3. Roteiro de testes

Abaixo estão os fluxos de teste recomendados para validar o comportamento do sistema:

Teste 1:

1. No teclado, digite 1 e 5 (o display deve mostrar 15).
2. Pressione A. O display começará a decrementar a cada 1 segundo.
3. Aguarde até o tempo chegar a 00. O sistema piscará os displays com “00” três vezes e retornará ao modo de espera.

Teste 2:

1. Digite um tempo qualquer (ex: 3 e 0).
2. Pressione A. O display começará a contagem.
3. Pressione B. O tempo deve parar.
4. Pressione B novamente. O display deve exibir “00”.

Teste 3:

1. Digite um tempo qualquer (ex: 6 e 7).
2. Pressione A. O display começará a contagem.
3. Pressione B. O tempo deve parar.
4. Pressione A. O tempo deve retornar a contagem.



Teste 4:

1. Digite um tempo qualquer (ex: 4 e 2).
2. Pressione A. O display começará a contagem.
3. Pressione C para abrir a porta. O tempo deve congelar imediatamente e o display deve exibir a mensagem "OP" (Open).
4. Tente pressionar A. O sistema deve ignorar o comando, pois a porta está aberta.
5. Pressione C novamente para fechar a porta. O tempo voltará a ser exibido e a contagem poderá ser retornada.

Teste 5:

1. Digite um tempo qualquer (ex: 9 e 0).
2. Pressione A. O display começará a contagem.
3. Pressione C para abrir a porta. O tempo deve congelar imediatamente e o display deve exibir a mensagem "OP" (Open).
4. Pressione B. O sistema zera o tempo interno da máquina, mas a porta continua fisicamente aberta.
5. O display continuará mostrando "OP". Para iniciar um novo processo, é obrigatório pressionar C para fechar a porta, voltar ao estado de espera ("00") e então digitar um novo tempo.

3. CONSIDERAÇÕES FINAIS

O desenvolvimento deste projeto cumpriu com êxito todos os requisitos propostos para a simulação do *firmware* de controle de um Microondas Automatizado. A adoção de uma arquitetura estritamente modular em linguagem Assembly MIPS revelou-se fundamental para garantir a estabilidade do sistema. Esta abordagem permite respostas rápidas ao nível do *hardware* simulado e o tratamento seguro de interrupções lógicas.

Adicionalmente à entrega funcional, a elaboração deste código consolidou na prática conceitos críticos de arquitetura de computadores, nomeadamente a manipulação de periféricos via *Memory-Mapped I/O* (através do Digital Lab Sim), o controle assíncrono de tempo através de chamadas de sistema (*syscall* 30) e a tradução de lógicas de alto nível para instruções de máquina. Importa referir que todo o sistema foi implementado, testado e validado utilizando o ambiente *MARS (MIPS Assembler and Runtime Simulator)* versão 4.5.

Entregáveis do Projeto

Junto a este relatório técnico e documentação de simulação, estão sendo entregues os seguintes arquivos-fonte que compõem o sistema:

- **main.asm:** Ponto de entrada e loop principal de execução.
- **logica.asm:** Máquina de estados finitos e controle lógico do micro-ondas.
- **teclado.asm:** Interface de leitura matricial e tratamento de entradas.
- **display.asm:** Controle e renderização visual dos displays de 7 segmentos.
- **timer.asm:** Sincronismo e temporização do relógio do sistema.

Suporte Técnico

Para suporte, favor contatar os desenvolvedores responsáveis:

- **Rafael Veronezi Ribeiro** - rafaelveronezir@gmail.com
- **Gabriel Augusto Baggio Varnier** - gabrielbaggio65@gmail.com